

🏠 ■ Sujets ■ IMAC2028 ■ S2 ■ [Sujet IMAC Island Viewer](#)



IMAC ISLAND VIEWER

Projet de programmation et algorithmique C++

- IMAC E3 - 2025-2026 -

Nombre de personnes par projet : 2 ou 3 (binôme ou trinôme)

Date de début : mardi 5 mai 2026

Date de rendu : mardi 9 juin 2026 à 23h

Soutenance : mercredi matin 17 juin 2026

⚠ L'usage d'IA est interdit pour ce projet

Introduction

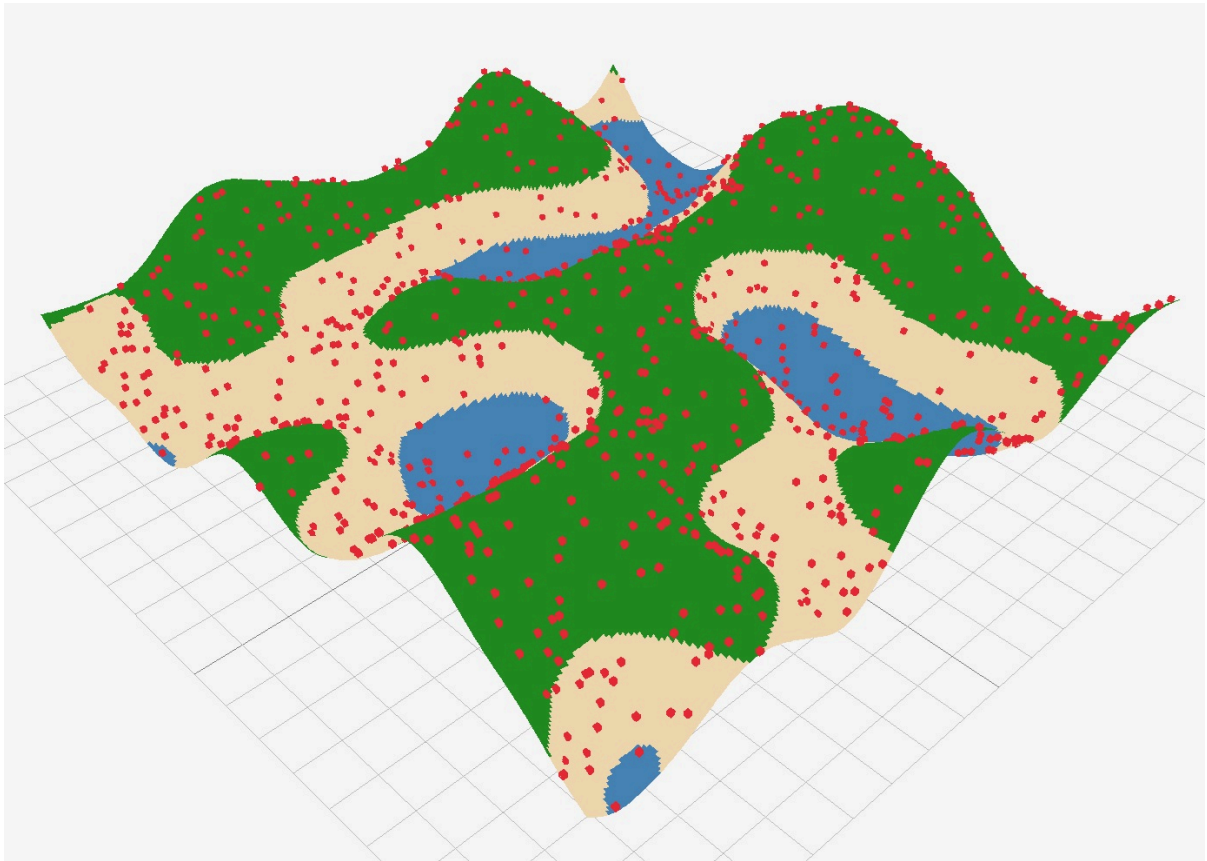
Pour mettre en pratique les connaissances acquises en programmation et en algorithmique, vous allez concevoir un générateur d'îles procédurales en C++.

Le projet s'appuie sur un squelette contenant déjà :

- une application C++ avec CMake
- un affichage 3D interactif d'une île (maillage + texture) utilisant **Raylib**
- des points/objets affichés sur le terrain
- des paramètres manipulables dans une UI simple (**ImGui**)

Vous pouvez télécharger le projet via ce lien : [IslandViewerStudents.zip](#)

Voilà ce qu'affiche l'application fournie:



Votre travail sera de remplacer les implémentations de génération de la heightmap (carte d'élévation), de la coloration du terrain et de la distribution des points par des algorithmes plus avancés :

- **Bruit fractal (du type bruit FBM)** pour la heightmap
- **Coloration** avec interpolation linéaire en fonction de la hauteur (valeur dans la heightmap)
- **Poisson disk sampling** pour la distribution des points

Vous devrez également ajouter au moins une fonctionnalité supplémentaire (2 pour les trinômes) pour améliorer la qualité de la génération et de l'application en général.

Cahier des charges

Voici les indications que vous devez respecter pour ce projet. Néanmoins, si vous souhaitez modifier/adapter certains points, vous devez au préalable faire valider ces changements.

Contraintes générales

- **Compilation** : Un système de compilation **CMake** devra être intégré à votre projet. Votre projet devra contenir tout ce qui permet de le compiler et de le faire fonctionner sur Linux ou Windows (plateforme de développement à préciser dans le rapport). En cas de

développement sur **macOS**, pensez à tester votre programme sur une autre machine afin que le projet compile sur windows ou linux.

- Le code doit être organisé en fichiers `.cpp` / `.hpp` cohérents
- Le projet doit être versionné avec **Git**
- Le rendu doit inclure le code source du projet avec vos ajouts ainsi qu'un fichier `README.md` faisant office de présentation et de rapport de votre travail.
- Un minimum de commits (de tous les membres du groupe) est attendu pour valider le projet (un commit de dernière minute avec tout le code ajouté d'un coup sera sanctionné)
- Le projet est à faire par **binôme** ou **trinôme**. Les **trinômes** devront réaliser une fonctionnalité supplémentaire par rapport aux binômes.
- Les **retards** de rendu seront également sanctionnés (en cas exceptionnel, il faut me prévenir à l'avance, et non le jour du rendu ou de la présentation)
- Si vous choisissez une fonctionnalité qui n'est pas dans la liste des suggestions d'amélioration, vous devez la faire valider avant de l'implémenter. De même, si vous souhaitez faire une amélioration qui est dans la liste mais que vous souhaitez l'adapter ou la faire évoluer d'une manière ou d'une autre, vous devrez en faire part pour la faire valider.
- ⚠ Pensez à ajouter un fichier `.gitignore` pour éviter de commiter des fichiers temporaires de compilation ou des fichiers binaires (principalement les fichiers dans les dossiers `build/` ou `bin/`)

Objectifs algorithmiques obligatoires

1) Bruit fractal

Implémenter une fonction de bruit fractal, en particulier du type **FBM (Fractal Brownian Motion)**, qui accumule plusieurs octaves d'un bruit de base (ex : **Perlin** (fourni dans le template), **Simplex**, etc) pour générer une carte de hauteur plus riche et plus réaliste.

Un bruit fractal est obtenu en **sommant** plusieurs "octaves" de bruit de base, chacune avec une **fréquence** et une **amplitude** différentes. Les paramètres principaux à contrôler sont :

- **Nombre d'octaves** : combien de couches de bruit accumulées
- **Lacunarity** : facteur multiplicateur de la fréquence pour chaque octave (ex: 2.0 signifie que chaque octave a une fréquence deux fois plus élevée que la précédente)
- **Gain** : facteur multiplicateur de l'amplitude pour chaque octave (ex: 0.5 signifie que l'amplitude de chaque octave est réduite de moitié par rapport à la précédente)

- **Seed** : valeur de départ pour la génération du bruit, permettant d'obtenir des résultats différents à chaque exécution ou de reproduire les mêmes résultats en utilisant la même seed.

REMARQUE

Une implémentation d'un bruit de **Perlin** avec seed est fournie dans le template. Vous pouvez implémenter votre propre version de bruit de base (ex : Simplex) si vous le souhaitez, et cela pourra être pris en compte comme amélioration (dans le cas où cette implémentation ne vient pas d'une ressource externe ou d'une librairie).

- **Scale** : facteur d'échelle pour contrôler la "taille" du bruit généré (généralement en multipliant les coordonnées d'entrée du bruit par ce facteur)

Pour résumer :

- implémentation d'une fonction (`octaveNoise`) qui prend en entrée une **position**, une fonction de bruit de base, des paramètres pour générer le bruit fractal (nombre d'octaves, lacunarity, gain, ...) et qui retourne une valeur de bruit fractal correspondante (dans une plage cohérente, par exemple `[-1,1]` ou `[0,1]`).

► `octaveNoise` et `std::function`

- exposition des paramètres dans l'interface pour permettre l'exploration visuelle

Vous trouverez de nombreuses ressources en ligne sur le sujet :

- <https://thebookofshaders.com/13/?lan=fr>
- <https://iquilezles.org/articles/fbm/>

2) Génération de heightmap et couleurs

Réécrire la génération de la carte de hauteur pour produire une île procédurale crédible.

Le but est de créer une carte d'élévation qui ressemble à une île, c'est-à-dire avec des zones d'élévation plus élevées au centre et des zones plus basses vers les bords (qui peuvent être sous le niveau de la mer). Pour cela, vous allez combiner (par une simple multiplication, par exemple) le bruit fractal généré précédemment avec un **masque** qui fera diminuer les valeurs d'élévation vers les bords de la carte.

Un masque radial (ex: fonction gaussienne centrée ou juste linéaire) est un choix simple et efficace.

Voilà un exemple de masque:



Enfin, il va falloir convertir les valeurs de hauteur en couleurs pour texturer le maillage de l'île. Par exemple, on peut utiliser du **bleu** pour les zones sous le niveau de la mer (inférieures à 0), du vert pour les plaines, du gris pour les montagnes, etc. Une interpolation linéaire simple entre différentes couleurs en fonction de la valeur de hauteur est attendue, mais vous pouvez aller plus loin en ajoutant des règles plus complexes (ex : mélange de textures en fonction de la pente, d'un type de biome, etc.).

Un minimum de 3 couleurs différentes est attendu avec au minimum une interpolation linéaire simple entre ces couleurs en fonction de la hauteur.

Attendus

- combiner le **bruit fractal** avec un **masque radial** pour créer une forme d'île.
- conversion des valeurs de bruit en couleurs pour texturer le maillage (**minimum** 3 couleurs différentes avec une interpolation linéaire simple)
- exposer les paramètres de génération dans l'interface

3) Distribution de points par Poisson disk sampling

Remplacer la génération aléatoire naïve de positions 2D par un algorithme de **Poisson disk sampling**, en particulier la version de [Bridson's](#).

L'idée de cet algorithme est de générer des points de manière à ce qu'ils soient **uniformément répartis** tout en respectant une **distance minimale** entre eux. Cela permet d'obtenir une

distribution plus naturelle et réaliste pour les objets placés sur l'île (ex: arbres, rochers, etc).

Fonctionnement de l'algorithme de [Poisson](#) disk sampling :

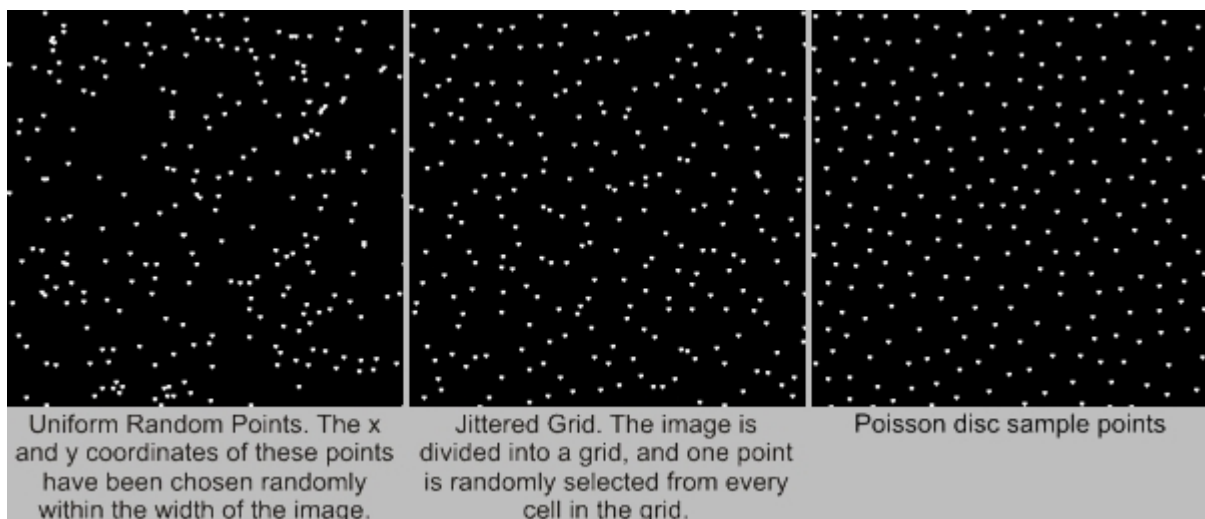
1. Choisir un point de départ aléatoire et l'ajouter à une liste de points actifs
2. Tant que la liste de points actifs n'est pas vide:
 - Choisir un point actif aléatoire
 - Générer jusqu'à k points candidats autour de ce point actif (dans un anneau entre r et $2r$, où r est la distance minimale)
 - Si un point candidat est à une distance d'au moins r de tous les points déjà générés, l'ajouter à la liste de points actifs et à la liste finale de points
 - Si après k essais aucun point candidat n'est valide, retirer le point actif de la liste

Dans la version de Bridson, une grille est utilisée pour accélérer la recherche de points voisins et vérifier rapidement si un point candidat est valide (pour éviter de parcourir tous les points générés à chaque fois).

Voilà deux vidéos qui expliquent l'algorithme de manière visuelle et intuitive :

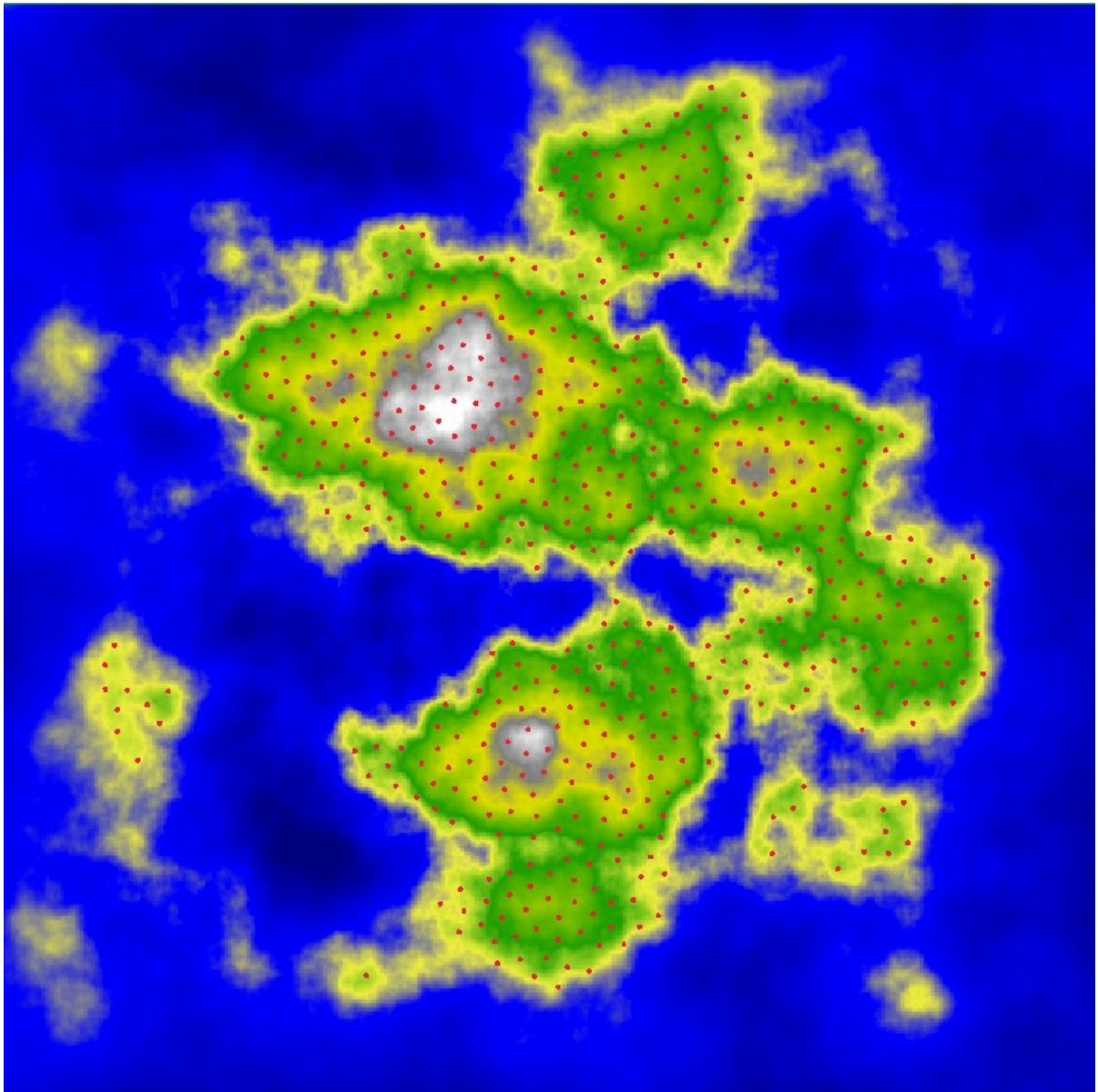
- [Coding Challenge #33: Poisson-disc Sampling](#)
- [Sebastian Lague: Procedural Object Placement](#)

Voilà une image qui illustre le résultat de cet algorithme en 2D en comparaison avec une génération aléatoire naïve:



Cette image provient de [cet](#) article, qui peut vous être utile pour comprendre et implémenter l'algorithme de Poisson disk sampling.

Voilà un exemple de résultat que vous pouvez viser dans votre application :



Attendus

- génération de points dans l'espace normalisé $[0,1] \times [0,1]$
- distance minimale entre deux points respectée
- paramètres contrôlables (rayon minimal, essais avant rejet, nombre de points maximum éventuel)

4) Placement des objets sur le terrain

Une fois les points 2D générés, il va falloir les projeter sur le terrain pour obtenir des positions 3D correspondantes à la surface de l'île. Le projet fournit déjà une fonction qui fait cela en échantillonnant la heightmap pour obtenir la hauteur correspondante à chaque point 2D.

Votre but est d'ajouter une possibilité de filtrage pour, par exemple, ne pas placer de points dans la mer (hauteur inférieure ou égale à 0) ou au sommet des montagnes (hauteur supérieure à un

certain seuil). Cela permettra d'obtenir un placement plus réaliste des objets sur l'île (ex : pas d'arbres dans la mer).

Attendus

- Ajouter une possibilité de filtrage des points générés en fonction de la hauteur du terrain (ex: pas de points dans la mer, ou au sommet des montagnes)

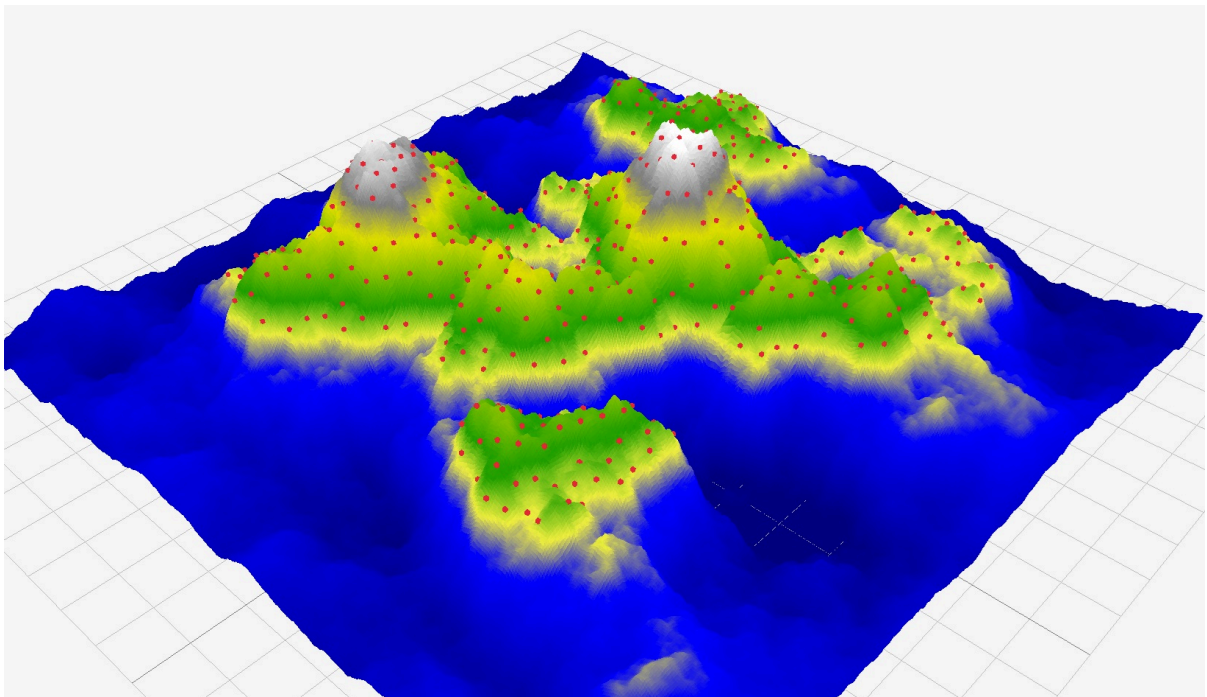
5) Améliorations

Une fois les objectifs principaux atteints, vous **devez** implémenter au moins une amélioration parmi les suggestions listées à la fin de ce document ou proposer votre propre idée d'amélioration (à valider).

Les **trinômes** devront obligatoirement réaliser **deux** améliorations (par rapport aux binômes qui doivent en faire au moins une).

Résultat

Voilà un exemple de résultat (mais n'hésitez pas à être créatifs et à proposer votre propre style d'île) :



Interface minimale attendue

Votre interface doit permettre au minimum :

- de régénérer la heightmap
- de régénérer le maillage
- de régénérer les positions
- de modifier les paramètres principaux (seed, échelle, résolution, etc).

L'objectif n'est pas une interface complexe, mais un outil de debug/exploration efficace.

Le projet fournit un exemple d'UI via **raylib** et **ImGui**.

Rapport

Le rapport doit contenir :

- sous quelle plateforme vous avez développé le projet (Linux, Windows, macOS)
- les choix algorithmiques faits
- les paramètres retenus et leur impact visuel
- les difficultés rencontrées et solutions
- quelques captures d'écran comparatives

Ajoutez enfin une partie "Post mortem" pour analyser le travail fourni : qu'est-ce qui a bien fonctionné, quels ont été les problèmes rencontrés, comment vous les avez surmontés, et ce que vous auriez fait différemment. Avec plus de temps, qu'est-ce que vous pourriez ajouter ? Comment s'est passée la répartition du travail dans le groupe ?

Le rapport doit être concis (par exemple 2 à 5 pages sans les illustrations).

Notation (indicative)

Voici un barème non définitif, à titre indicatif :

- Implémentation du bruit fractal : **/2**
- Génération de heightmap (couleurs, masque "radial", etc.) : **/4**
- Poisson disk sampling : **/4**
- Améliorations demandées : **/4**
- BONUS (améliorations supplémentaires ou plus avancées, originalité, etc) : **/3**
- Qualité du code : **/2**
- Rapport : **/2**

Total : **/21** qui sera ramené à 20 pour la notation finale

Améliorations suggérées

Voilà quelques idées d'améliorations que vous pouvez implémenter. Ce sont des suggestions, n'hésitez pas à être créatifs et à proposer vos propres idées (à valider) :

- Import d'un mesh 3D pour le placement d'objets (ex: un modèle d'arbre) plutôt que des cubes simples (il existe des exemples [ici](#) ou [ici](#) de chargement de modèles dans raylib). Les modèles 3D utilisés doivent être libres de droits ou alors réalisés par vous même.
- Gestion de palettes de couleurs pour la coloration du terrain
- Couleurs/textures plus avancées (ex: mélange en fonction de la pente, d'un type de biome, etc).
- Ajout de plusieurs types de bruits ([Simplex](#), Worley, etc) **et** possibilité de les combiner.
- Placement des objets amélioré (variation de la taille, de la rotation, etc)
- Génération de différentes formes d'îles (ex: île en croissant, île avec un lac intérieur, etc) via des masques plus complexes (au moins 3 formes différentes).
- Distribution de plusieurs types d'objets avec conditions (ex: arbres sur les pentes douces, rochers sur les pentes raides, etc). Utilisation de plusieurs palettes de couleurs.
- Ajout de biomes (ex: plage, forêt, montagne) qui peuvent avoir différents types d'impact sur la génération (différentes couleurs, différentes règles de distribution d'objets, différents types de bruit, etc).
- Détection des îles (pouvoir détecter les zones connectées sur une même île sans passer par la "mer" (niveau d'élévation inférieur ou égal à 0)) afin de faire varier les couleurs, le placement d'objets, ou ajouter des ponts (plus difficile). Ce type d'algorithme s'appelle "connected components detection". Vous pouvez trouver un article à ce sujet [ici](#).

Conseils de réalisation

- Commencez par valider chaque brique séparément avant de les combiner:
 - accumulation octaves
 - masque d'île
 - coloration en fonction d'une valeur de hauteur
 - Poisson disk sampling
- Privilégiez la lisibilité et la stabilité à la complexité inutile

- Faites des commits **fréquents** et bien nommés pour documenter votre progression (exemple: un commit par amélioration)